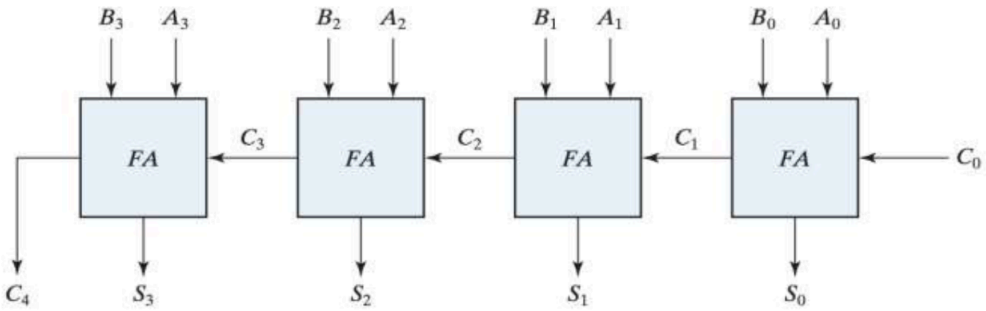
Because a full adder can only do one column bit addition for a multibit binary number, chaining must occur for the full addition to be processed within a computer.

Remember that a full adder takes three one-bit inputs and produces two one-bit outputs.

- Inputs: x and y (the two bits we want to add) and z (a carry coming in from the position to the right).
- Outputs: S (the sum bit for this position) and C (the carry going out t o the next position on the left.
-

When you add column by column, the column to your right might hand you a leftover carry, so each column really adds three bits, not two.

When you add two multi-digit numbers by hand, you don't add the whole thing in one step. You add the rightmost column, write the sum digit, carry the overflow into the next column, and repeat moving left. A binary adder does the same thing, except each "column" is a single bit and each column has its own full adder.



The above circuit is called a **Ripple Adder**. It is a circuit of chained full adders together for each bit position. There are four adders, so the circuit is computing the binary addition of 4 bit numbers.

How the wiring works:

1. The rightmost adder FA_0 receives A_0, B_0, and an input carry C_0. Since nothing comes in from the right, we set $C_0 = 0$.
2. FA_0 produces sum bit S_0 and a carry C_1.
3. That carry C_1 becomes the carry-in for the next adder FA_1, which also gets A_1 and B_1. It produces S_1 and carry C_2.
4. This keeps going. Each adder hands its carry one position to the left.
5. The final adder FA_3 produces S_3 and the last carry C_4, which is the overall carry-out of the whole addition.

The output of the circuit is the 5-bit result $C_4S_3S_2S_1S_0$.

Example) A = 1011 and B = 0011. 11 + 3 should give us 14. We work one column at a time, right to left. At each column we add three bits: the A bit, B bit, and C bit.

Position i	3	2	1	0
Carry in C_i	0	0	1	0
Augend A_i	1	0	1	1
Addend B_i	0	0	1	1
Sum S_i	1	1	1	0
Carry out C_{i+1}	0	0	1	1

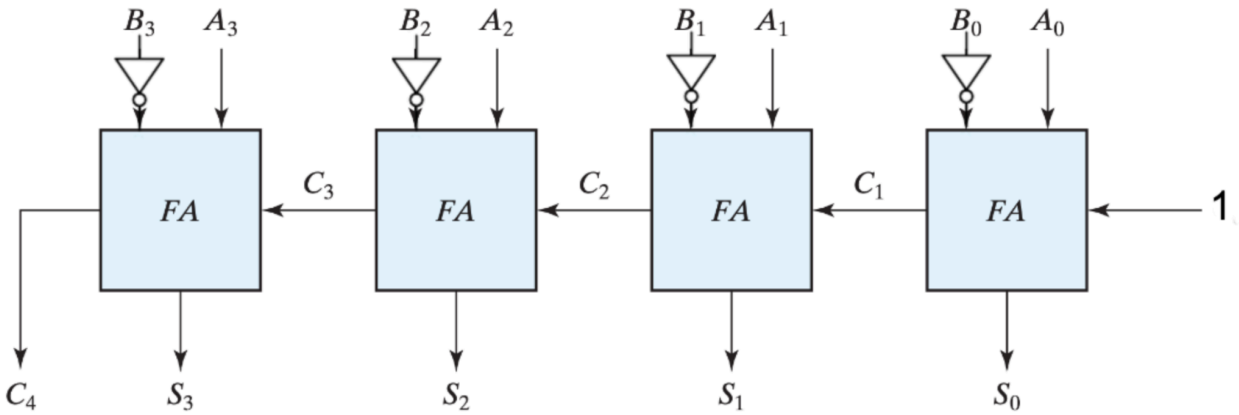
Real gates are not instant. When a signal enters a gate, it takes a tiny amount of time, referred to as the **propagation delay,** before the output settles to the correct value. In a ripple-carry adder, this creates a problem. Because the carry must ripple all the way through the chain before the leftmost bit is generated, it will take some time for the binary addition to be complete. The greater the bits you are adding, the more adders necessary, and the longer it will take. A common fix is the implementation of the **carry-lookahead logic**, which computes the carries in parallel instead of waiting for them to ripple.

Subtracting a number is the same as adding its negative, and in binary we represent negatives using two's complement. This means we can reuse the adder we already built to create a **binary subtractor**. To compute A - B:

1. Take the 1's complement of B (0 becomes 1, 1 becomes 0).
2. Add 1 to that result. This gives the 2's complement of B , which represents -B.
3. Add this to A. The result is A + (-B) = A - B.

To create a subtractor from the ripple adder:

1. **Flip the bits of B.** The 1's complement just means inverting each bit of B. An inverter (NOT gate) does exactly this.
2. **Add the extra 1.** Remember the rightmost adder FA_0 had its carry-in C_0 fixed at 0? We can sneak in the "+1" for free by setting $C_0 = 1$ instead. The adder happily adds that 1 into the least significant position.

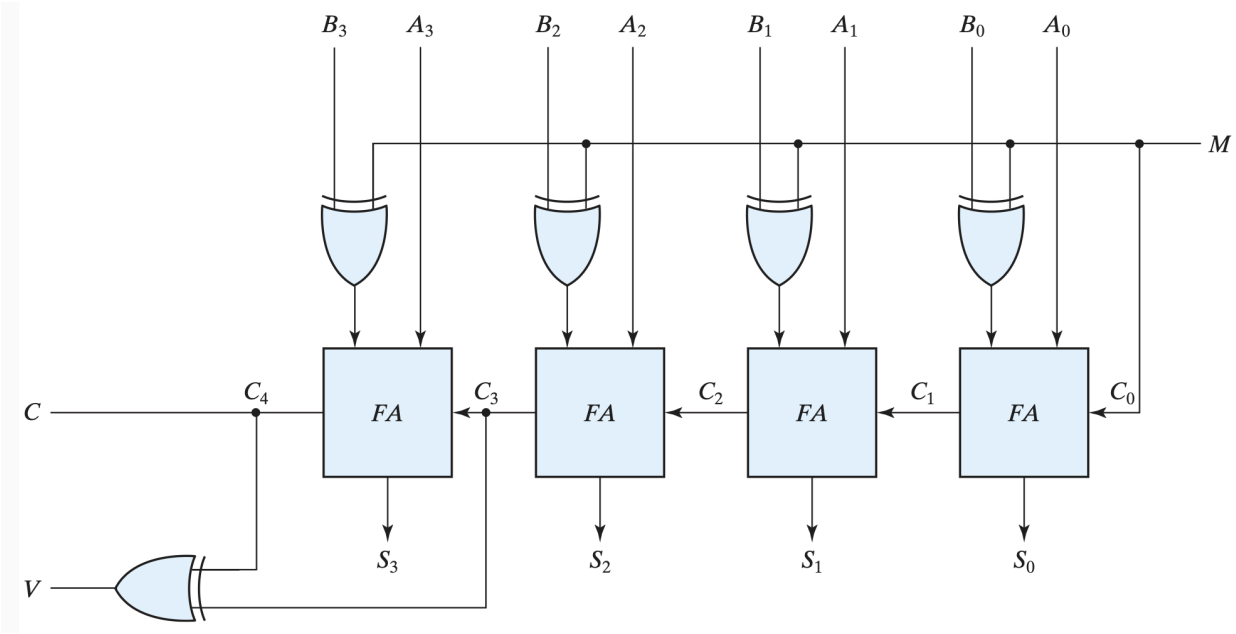


We can make a single circuit that adds when we want and subtracts when we want, controlled by one signal. We use the exclusive-OR (XOR) gate as a "controllable inverter." The signal will tell the XOR gates to invert B and change the first carry into 1 for subtraction, or leave B alone and keep the first carry as 0.

Remember these key properties:

$B \oplus 0 = B$ (passes B through unchanged)

$B \oplus 1 = \overline{B}$ (flips B)



- When $\mathbf{M = 0}$: each B bit passes through unchanged and $C_0 = 0$. The circuit computes $A + B$. **It's an adder.**
- When $\mathbf{M = 1}$: each B bit is inverted and $C_0 = 1$. The circuit computes $A + (\overline{B}) + 1 = A - B$. **It's a subtractor.**

The above circuit can perform both signed and unsigned arithmetic. It also incorporates new outputs, C and V. For unsigned arithmetic, only that C is what matters:

- When addition: if C = 1, that means unsigned overflow has occurred.
- When addition: if C = 0, that means no unsigned overflow has occurred.
- When subtraction: if C = 1, that means A > B
- When subtraction: if C = 0, that means A < B

In signed arithmetic, only V is what matters:

- V = 1 means overflow has occurred.
- V = 0 means no overflow has occurred.